

如何实现15分钟未支付自动取消订单？

在开发中，往往会遇到一些关于延时任务的需求。

例如

- 生成订单 15 分钟未支付，则自动取消
- 生成订单 60 秒后，给用户发短信

对上述的任务，我们给一个专业的名字来形容，那就是延时任务。那么这里就会产生一个问题，这个延时任务和定时任务的区别究竟在哪里呢？一共有如下几点区别

定时任务有明确的触发时间，延时任务没有

定时任务有执行周期，而延时任务在某事件触发后一段时间内执行，没有执行周期

定时任务一般执行的是批处理操作是多个任务，而延时任务一般是单个任务

下面，我们以判断订单是否超时为例，进行方案分析

方案 1：数据库轮询

思路

该方案通常是在小型项目中使用，即通过一个线程定时的去扫描数据库，通过订单时间来判断是否有超时的订单，然后进行 update 或 delete 等操作



单体应用：利用Spring-Task或者Quartz等单机任务调度工具定时执行



集群环境：比如利用分布式任务调度工具XXL-JOB执行分布式任务调度



优点

简单易行，支持集群操作，小项目首选

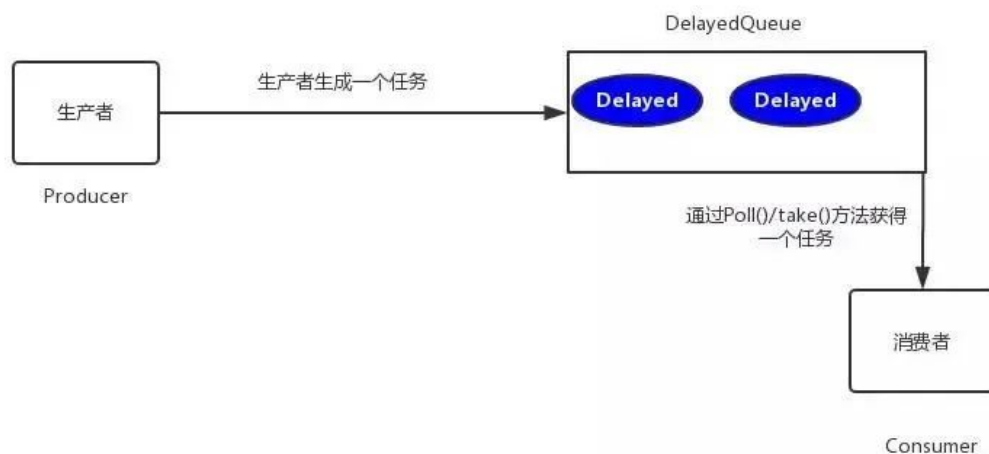
缺点

- 对服务器内存消耗大
- 存在延迟，比如你每隔 3 分钟扫描一次，那最坏的延迟时间就是 3 分钟
- 假设你的订单有几千万条，每隔几分钟这样扫描一次，数据库损耗极大
- xxl-job不是所有项目都有

方案 2：JDK 的延迟队列

思路

该方案是利用 JDK 自带的 DelayQueue 来实现，这是一个无界阻塞队列，该队列只有在延迟期满的时候才能从中获取元素，放入 DelayQueue 中的对象，是必须实现 Delayed 接口的，DelayedQueue 实现工作流程如下图所示



其中 Poll():获取并移除队列的超时元素，没有则返回空

take():获取并移除队列的超时元素，如果没有则 wait 当前线程，直到有元素满足超时条件，返回结果。

优点

效率高,任务触发时间延迟低。

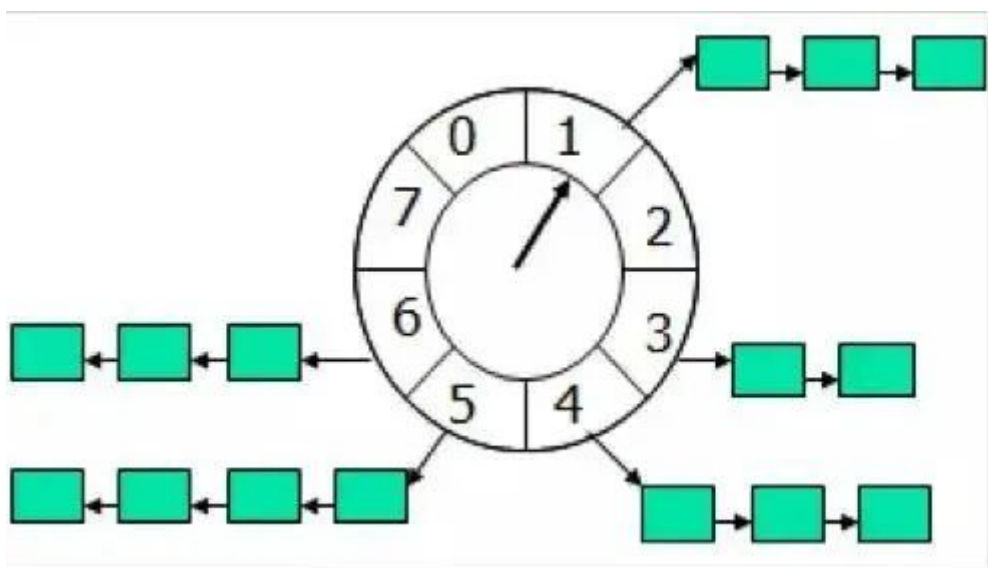
缺点

- 服务器重启后，数据全部消失，怕宕机
- 集群扩展相当麻烦
- 因为内存条件限制的原因，比如下单未付款的订单数太多，那么很容易就出现 OOM 异常
- 代码复杂度较高

方案 3：时间轮算法

思路

先上一张时间轮的图



时间轮算法可以类比于时钟，如上图箭头（指针）按某一个方向按固定频率轮动，每一次跳动称为一个 tick。这样可以看出定时轮由个 3 个重要的属性参数，ticksPerWheel（一轮的 tick 数），tickDuration（一个 tick 的持续时间）以及 timeUnit（时间单位），例如当 ticksPerWheel=60，tickDuration=1，timeUnit=秒，这就和现实中的始终的秒针走动完全类似了。

如果当前指针指在 1 上面，我有一个任务需要 4 秒以后执行，那么这个执行的线程回调或者消息将会被放在 5 上。那如果需要在 20 秒之后执行怎么办，由于这个环形结构槽数只到 8，如果要 20 秒，指针需要多转 2 圈。位置是在 2 圈之后的 5 上面 ($20 \% 8 + 1$)

优点

效率高,任务触发时间延迟时间比 delayQueue 低，代码复杂度比 delayQueue 低。

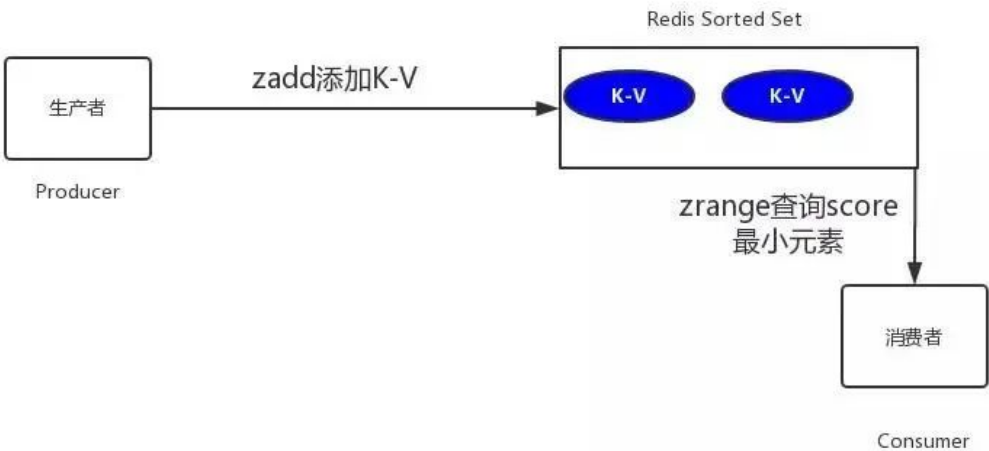
缺点

- 服务器重启后，数据全部消失，怕宕机
- 集群扩展相当麻烦
- 因为内存条件限制的原因，比如下单未付款的订单数太多，那么很容易就出现 OOM 异常

方案 4: redis 缓存

思路一：定时轮询有序集合

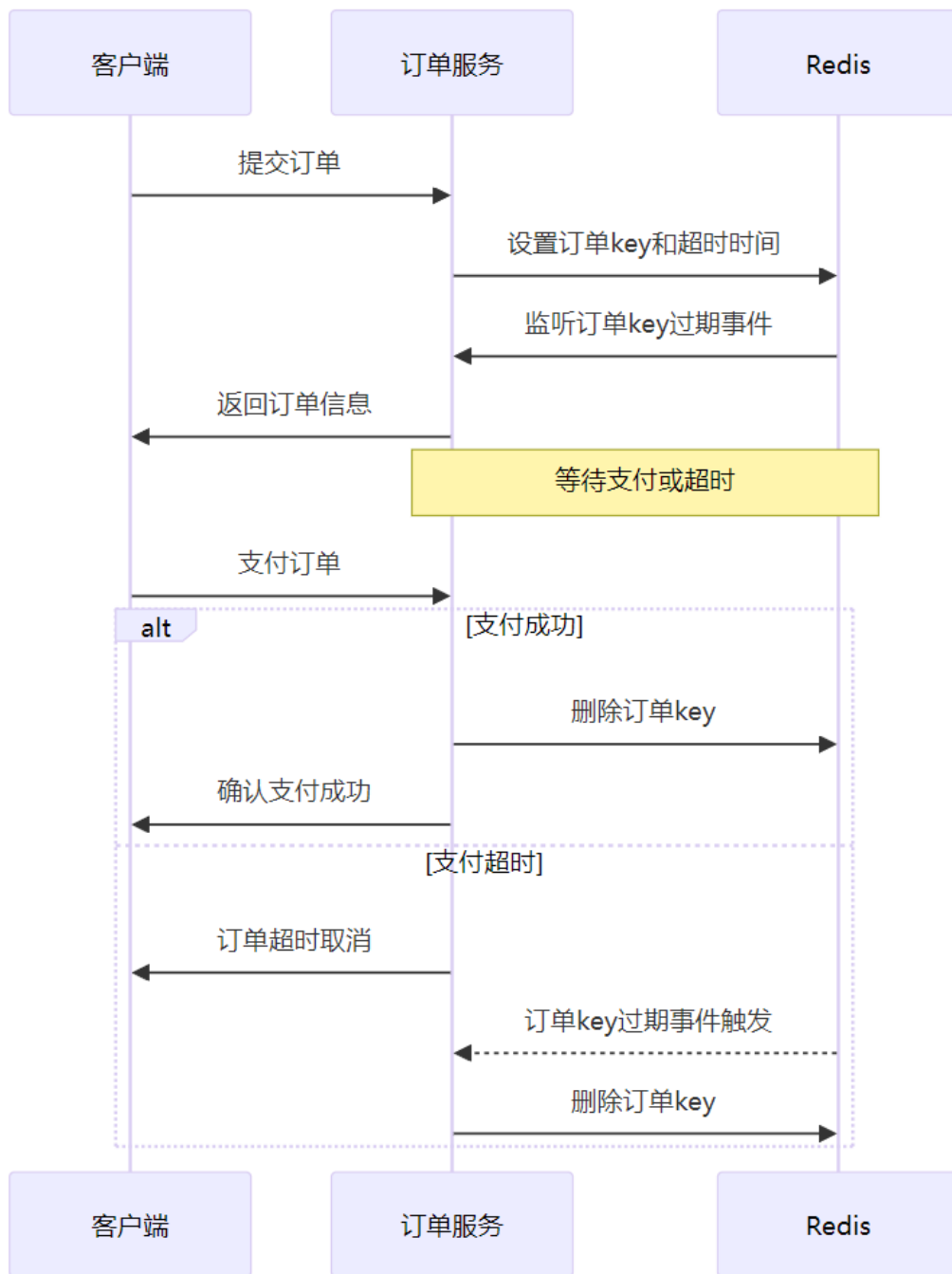
利用 redis 的 zset是一个有序集合，每一个元素(member)都关联了一个 score，通过 score 排序来取集合中的值，我们将订单超时时间戳与订单号分别设置为 score 和 member，系统扫描第一个元素判断是否超时，具体如下图所示



```
1 1525096202813ms:OID00000000订单生成
2 1525096202818ms:OID00000001订单生成
3 1525096202824ms:OID00000002订单生成
4 1525096202826ms:OID00000003订单生成
5 1525096202830ms:OID00000004订单生成
6 1525096202834ms:OID00000005订单生成
7 1525096202839ms:OID00000006订单生成
```

思路二：使用Redis key过期监听

该方案使用 redis 的 Keyspace Notifications，就是利用该机制可以在 key 失效之后，提供一个回调，实际上是 redis 会给客户端发送一个消息，就是pub/sub机制，需要 redis 版本 2.8 以上，但是pub/sub 机制存在一个硬伤：Redis 的发布/订阅目前是即发即弃(fire and forget)模式的，因此无法实现事件的可靠通知。也就是说，如果发布/订阅的客户端断链之后又重连，则在客户端断链期间的所有事件都丢失了。因此，思路二不是太推荐。当然，如果你对可靠性要求不高，可以使用。

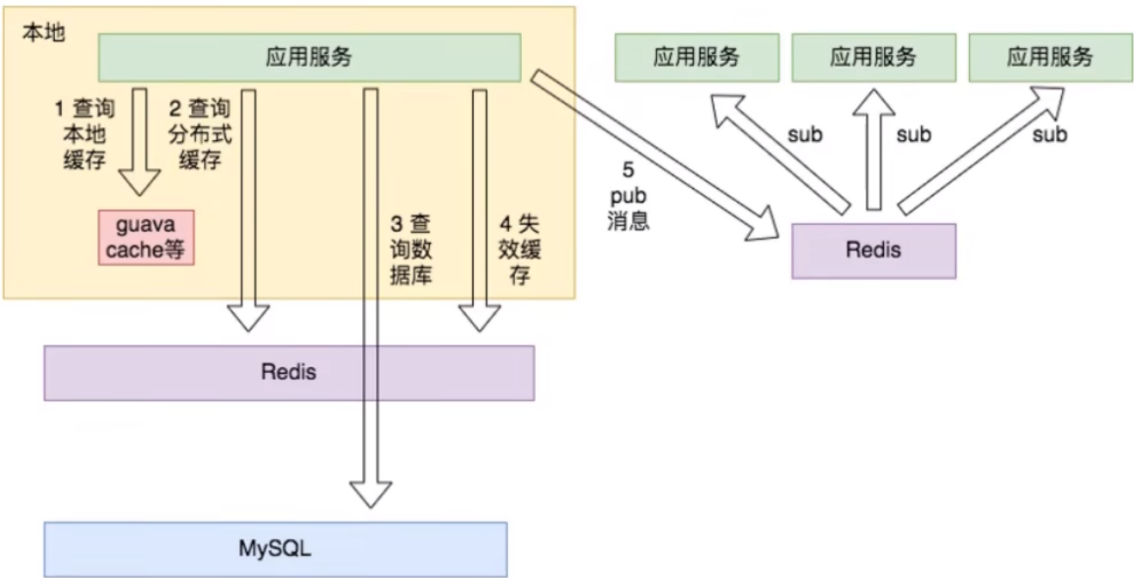


思路三：Redis 6 客户端缓存监听方案

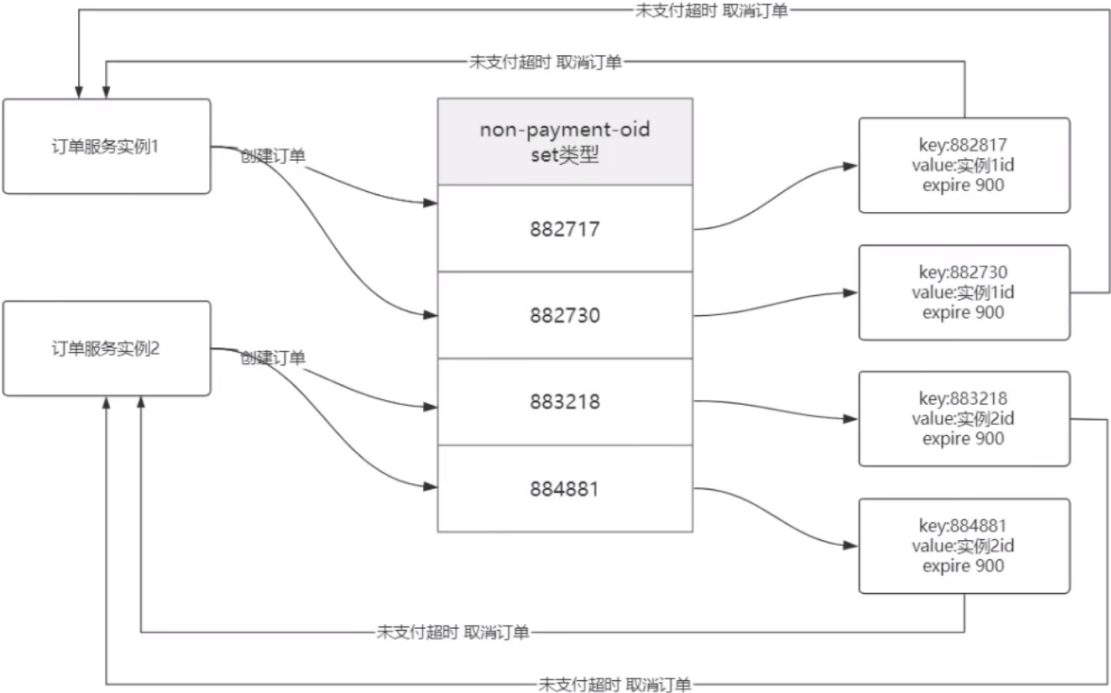
普通模式。在这个模式下，实例会在服务端记录客户端读取过的 key，并监测 key 是否有修改。一旦 key 的值发生变化，服务端会给客户端发送 invalidate（失效）消息，通知客户端缓存失效了。在使用普通模式时，有一点你需要注意一下，服务端对于记录的 key 只会报告一次 invalidate 消息，也就是说，服务端在给客户端发送过一次 invalidate 消息后，如果 key 再被修改，此时，服务端就不会再次给客户端发送 invalidate 消息。只有当客户端再次执行读命令时，服务端才会再次监测被读取的 key，并在 key 修改时发送 invalidate 消息。这样设计的考虑是节省有限的内存空间。毕竟，如果客户端不再访问这个 key 了，而服务端仍然记录 key 的修改情况，就会浪费内存资源。

广播模式：在这个模式下，服务端会给客户端广播所有 key 的失效情况，不过，这样做了之后，如果 key 被频繁修改，服务端会发送大量的失效广播消息，这就会消耗大量的网络带宽资源。所以，在实际应用时，我们会让客户端注册希望跟踪的 key 的前缀，当带有注册前缀的 key 被修改时，服务端会把失效消息广播给所有注册的客户端。和普通模式不同，在广播模式下，即使客户端还没有读取过 key，但只要

它注册了要跟踪的 key，服务端都会把 key 失效消息通知给这个客户端。。我们在实际应用时，会给同一业务下的 key 设置相同的业务名前缀，所以，我们就可以非常方便地使用广播模式。



实现细节



优点

及时有效，具备主动推送功能

由于使用 Redis 作为消息通道，消息都存储在 Redis 中。如果发送程序或者任务处理程序挂了，重启之后，还有重新处理数据的可能性。

做集群扩展相当方便，哪个实例发起的订单哪个实例负责取消

时间准确度高

缺点

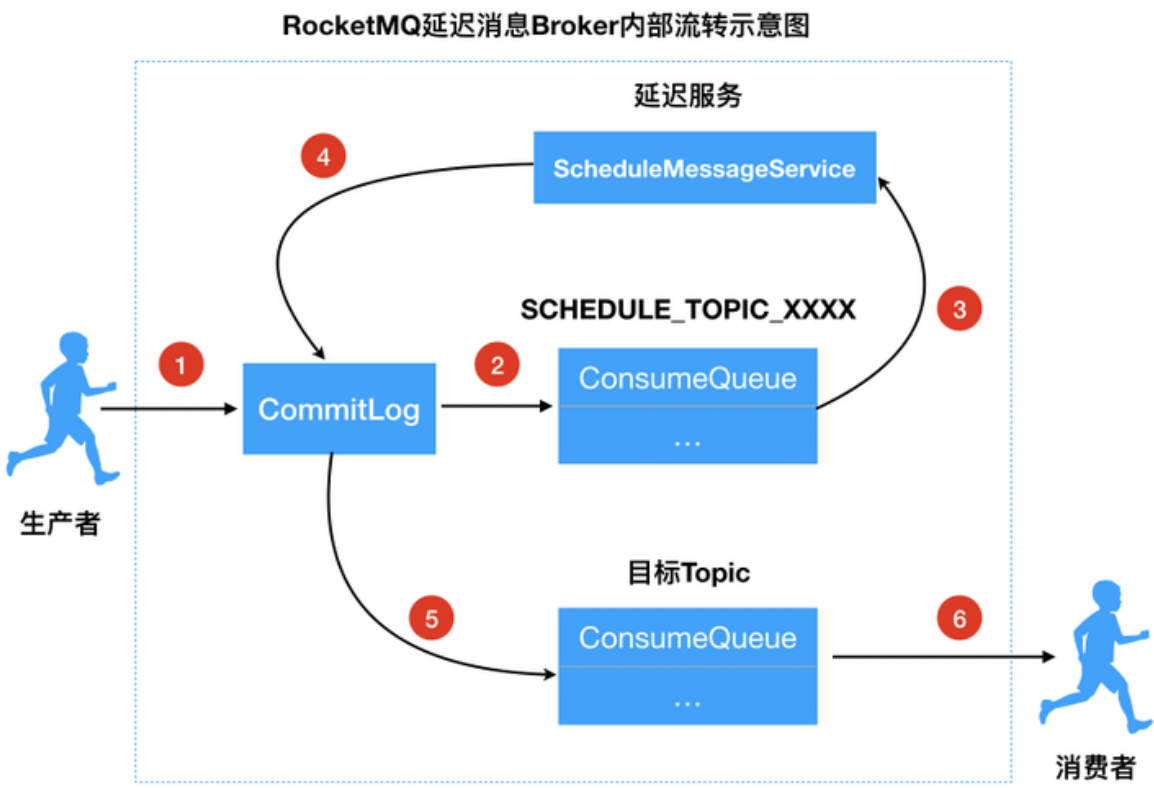
- 1. 需要额外进行 redis 维护
 - 2. 基于长链接，连接重连后客户端缓存监听机制会失效，需要手动补偿
 - 3. 实例数量发生变化，需要重新做分配
- Redis6+新特性，需要升级Redis

方案 5：消息队列

RocketMQ发送消息之延迟消息

延迟消息是指消息发送后，消费者要在一定时间后，或者指定某个时间点才可以消费。在没有延迟消息时，基本的做法是基于定时计划任务调度，定时发送消息。在 RocketMQ中只需要在发送消息时设置延迟级别即可实现。

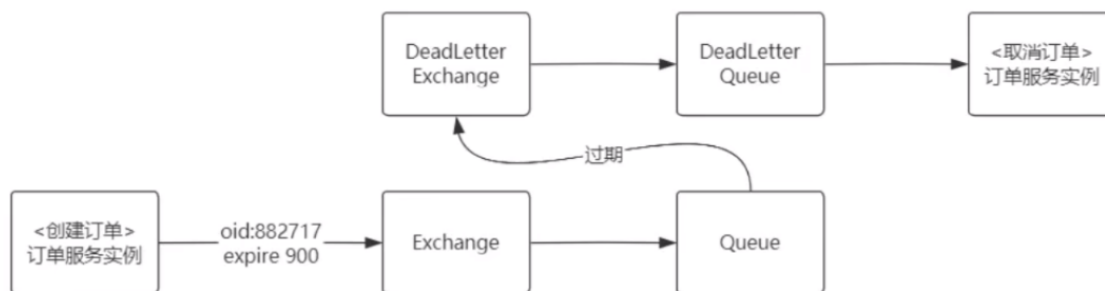
1s 5s 10s 30s 1m 2m 3m 4m 5m 6m 7m 8m 9m 10m 20m 30m 1h 2h



RocketMQ延迟队列



RabbitMQ及其他利用死信队列



优点

消息及时投递，集群友好，代码量小，不需要额外集群调整，也不绑定具体实例

缺点

必须要有MQ，而且要玩的溜

需要额外关注幂等性等MQ自身引起的新问题